INTELLIGENT AGENTS

Evaluating AI Agents

Estimate the trajectory of momentum, not the exact destination.

TA: Akhil Chintalapati

Citation: *Demystifying evals for AI agents*. Engineering at Anthropic. Retrieved February 25, 2026, 9:26 AM (ET), from https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents. Used for educational purposes only; all rights remain with Anthropic.

Why Bother with Evals?

Without them, you're always reacting - never improving confidently

Without Evals

User reports something broke

You manually reproduce the bug

You fix it and hope nothing regressed

Repeat indefinitely…

→ Can't distinguish real regressions from noise

→ Can't test 100s of scenarios before shipping

→ Can't measure improvement - only feel it

With Evals

Failures become test cases automatically

Run hundreds of checks before every release

Metrics replace guesswork - show progress

Catch regressions before users do

→ Accelerate model upgrades (days vs weeks)

→ Give research teams a hill to climb

→ Align the team on what 'good' means

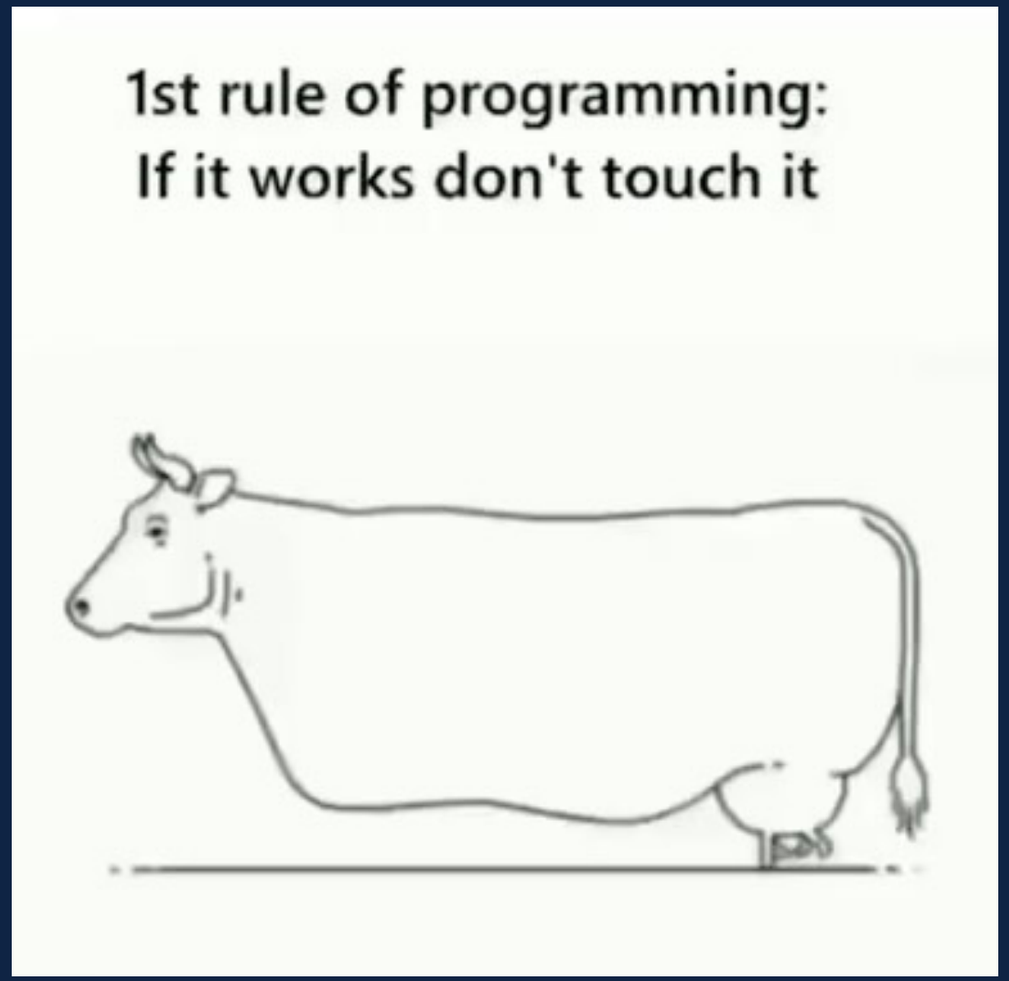

Figure: Wide-Coder Doctrine - Intervene Only When Entropy Wins.

The Vocabulary of an Eval

Six terms

Task

One test: inputs + success criteria

Trial

A single attempt at a task. Run many - LLMs are non-deterministic

Grader

Logic that scores a dimension of the output. Tasks can have several

Transcript

Full record: every API call, tool use, reasoning step

Outcome

End state in the environment - not what the agent said, what actually happened

Harness

Infrastructure that runs tasks, calls tools, records steps, aggregates scores

Three Types of Graders

Each measures something different

Code-Based

String / regex match
Unit tests (pass/fail)
Tool-call verification
Transcript stats (turns, tokens)
Static analysis (lint, mypy)

✓ Fast, cheap, reproducible
✗ Brittle - valid solutions may fail

Model-Based (LLM Judge)

Rubric-based scoring
Natural-language assertions
Pairwise comparison
Reference-based grading

✓ Handles nuance & open-ended output
✓ Scales to many tasks
✗ Non-deterministic, needs calibration

Human

Expert spot-check
Crowdsourced judgment
A/B preference studies

✓ Gold standard - matches real users
✓ Calibrates your LLM judges
✗ Slow, expensive, doesn't scale
→ Use sparingly but regularly

Matching Graders to Agent Types

Same grader toolkit

Coding Agents

Unit tests = primary signal (does it run?)

Static analysis: lint, type errors, security

Transcript: turns taken, tools used correctly

LLM rubric: code quality, style, over-engineering

Key insight: software has ground truth - lean on it

Conversational Agents

State check: was the ticket resolved in DB? (outcome)

Transcript constraint: completed in ≤k turns?

LLM rubric: empathy, clarity, policy grounding

Tool verification: right tools called?

Key insight: the interaction quality IS the product

Research Agents

Groundedness: every claim has a source

Coverage: key facts from reference answer included

Source quality: authoritative, not just first result

LLM rubric for coherence & completeness

Key insight: calibrate LLM judges with human experts -

research quality is inherently subjective

Non-Determinism: pass@k vs pass^k

Same agent, same task - results vary. These two metrics tell different stories.

pass@k - At Least One Success

P(agent succeeds ≥1 time in k tries)

As k ↑, score rises toward 100%
More shots on goal = higher odds of 1 success

Use when: one correct answer is enough
→ Coding: find a working solution
→ k=5 coverage evals, research synthesis

pass@1 = per-trial success rate
pass@10 → accounts for lucky & unlucky runs

pass^k - All k Must Succeed

P(agent succeeds every time in k tries)

As k ↑, score drops toward 0%
Demanding consistency is a harder bar

Example: 75% per-trial × 3 trials
→ pass^3 = $(0.75)^3 \approx 42\%$

Use when: users need reliable behavior
→ Customer-facing conversational agents
→ Safety-critical actions

At k=1 they're identical. By k=10 they tell opposite stories: pass@k → 100%, pass^k → 0%

From Zero → Evals You Can Trust

8 steps - a roadmap from Anthropic teams

0 Start Early

20–50 tasks beats waiting for 200. Evals get harder to build the longer you wait.

1 Mine Manual Checks

Convert your pre-release checks & bug tracker issues into test cases first.

2 Write Unambiguous Tasks

Two experts should agree on pass/fail - independently. If not, rewrite the task.

3 Build Balanced Sets

Test both when behavior should occur and when it shouldn't. One-sided → one-sided agent.

4 Isolate Environments

Each trial starts clean. Shared state (stale files, caches) creates correlated & false failures.

5 Grade Thoughtfully

Grade outcomes, not paths. Partial credit for partial success. Calibrate LLM judges.

6 Read the Transcripts

You'll find bugs in your graders, not the agent. Failures should feel fair.

7 Maintain Live Suites

Capability evals graduate to regression suites. Watch for saturation (100% = useless).

Thinking Like an Eval Designer

Principles over rules - the mindset behind good evals

Grade Outcomes Too, Not Only Paths

Agents find creative solutions eval writers didn't anticipate.

Checking "did it call tools in this exact order?" punishes creativity.

Ask: did it produce the right result? Not: did it follow my script?

Balance Both Directions

Testing only "should it act?" → agent acts on everything.

Test when action is correct AND when it isn't.

Example: search evals need "should search" AND "should not search" cases.

Partial Credit Matters

An agent that identifies the problem correctly but fails the last step

is meaningfully better than one that fails immediately.

Binary pass/fail hides this gradient of success.

0% pass@k ≈ Broken Eval, Not Broken Agent

With frontier models, consistent total failure usually means your task

spec is ambiguous, your grader has a bug, or the environment is broken.

Always create a reference solution that proves the task is solvable.

Beware Eval Saturation

100% pass rate means you're tracking regressions only - no improvement signal.

Capability evals should start challenging and grow harder over time.

Graduated evals become regression suites; build new capability evals.

Evals Are a Communication Tool

Writing tasks forces teams to agree on what "good" means before building.

Product managers, researchers, and engineers converge on concrete specs.

The best eval tasks are written by those closest to users.

Evals Are One Layer - Not the Whole Story

Like the Swiss Cheese Model: no single method catches everything

Automated Evals

Pre-launch, CI/CD

Pros:
+ Fast, reproducible, scalable
+ Run on every commit
+ No user impact

Cons:
− Needs upfront investment
− Can miss real-world edge cases

Prod Monitoring

Post-launch

Pros:
+ Real behavior at scale
+ Ground truth signal
+ Catches distribution drift

Cons:
− Reactive - users impacted first
− Signals can be noisy

A/B Testing

Sufficient traffic

Pros:
+ Measures real user outcomes
+ Statistically rigorous

Cons:
− Weeks to significance
− Only tests what you deploy

Manual Transcripts

Ongoing (weekly)

Pros:
+ Builds intuition
+ Catches subtle quality gaps
+ Calibrates what 'good' means

Cons:
− Doesn't scale
− Inconsistent coverage

Human Studies

Calibration

Pros:
+ Gold-standard judgment
+ Calibrates LLM judges
+ Handles subjective tasks

Cons:
− Expensive & slow
− Hard to run frequently

Key Takeaways

Think in these terms, not in rigid rules

Start early, start small.

20 real tasks > waiting for the perfect 200. Evals compound in value.

Grade what matters.

Outcomes over paths. Partial credit over binary. Balance both directions.

Read your transcripts.

The transcript shows whether the agent failed - or your grader did.

Combine all three grader types.

Code for speed, LLM for nuance, human for calibration and ground truth.

Non-determinism is not noise.

pass@k and pass^k are deliberate choices - pick the one your product needs.

Evals define team alignment.

Writing tasks forces everyone to agree on what 'good' means before shipping.

"Teams that invest early find: development accelerates as failures become test cases, test cases prevent regressions, and metrics replace guesswork." - Anthropic

TASTES GREAT!